

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - STL 개념정리	date

2026.06.08

C++ STL 개념정리

핵심

STL은 C++에서 자주 쓰는 자료구조와 알고리즘을 미리 만들어둔 표준 라이브러리이다. STL을 쓰는 이유는 배열, 정렬, 탐색, 스택, 큐, map 같은 기능을 직접 만들지 않고 검증된 방식으로 빠르게 사용하기 위해서이다.

왜 필요한가

프로그램을 만들다 보면 데이터를 저장하고, 정렬하고, 찾고, 추가하고, 삭제하는 일이 계속 나온다. 이 기능을 매번 직접 구현하면 시간이 오래 걸리고 오류가 생기기 쉽다. STL은 이런 반복 작업을 줄이고, 더 안정적인 코드를 작성하기 위해 필요하다.

STL의 구성

STL은 크게 컨테이너, 반복자, 알고리즘으로 나눌 수 있다.

컨테이너

데이터를 저장하는 자료구조이다. 예: `vector`, `list`, `stack`, `queue`, `map`, `set`

반복자

컨테이너 안의 원소를 가리키는 도구이다. 포인터와 비슷하게 동작한다.

알고리즘

정렬, 탐색, 개수 세기 같은 기능이다. 예: `sort`, `find`, `count`

vector

`vector`는 크기가 자동으로 늘어나는 배열이다.

```
vector<int> numbers;  
numbers.push_back(10);  
numbers.push_back(20);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 필요한가

일반 배열은 크기를 미리 정해야 한다. 데이터 개수가 변할 수 있는 상황에서는 배열만으로 관리하기 어렵다. `vector`는 필요할 때 크기가 늘어나므로 책 목록, 점수 목록, 데이터 목록 관리에 유리하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - STL 개념정리	date

list

'list'는 연결 리스트이다.

```
list<int> numbers;
numbers.push_back(10);
numbers.push_front(5);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 필요한가

중간 삽입과 삭제가 자주 일어나는 경우 'list'가 유리할 수 있다. 단, 인덱스로 바로 접근하는 것은 'vector'보다 불편하다.

stack

'stack'은 나중에 들어간 데이터가 먼저 나오는 구조이다.

```
stack<int> s;
s.push(10);
s.push(20);
s.pop();
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 필요한가

가장 최근에 들어온 작업을 먼저 처리해야 할 때 필요하다.

예시:

- 뒤로가기
- 괄호 검사
- 함수 호출 구조
- 되돌리기 기능

queue

'queue'는 먼저 들어간 데이터가 먼저 나오는 구조이다.

```
queue<int> q;
q.push(10);
q.push(20);
q.pop();
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - STL 개념정리	date

왜 필요한가

들어온 순서대로 처리해야 할 때 필요하다.

예시:

- 대기줄
- 프린터 출력 순서
- 작업 예약
- 순서대로 처리하는 요청

map

`map`은 key와 value를 한 쌍으로 저장한다.

```
map<string, int> score;  
score["Kim"] = 90;  
score["Lee"] = 85;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 필요한가

값을 번호가 아니라 이름, 제목, 아이디 같은 key로 찾고 싶을 때 필요하다.

예시:

- 이름으로 점수 찾기
- 책 제목으로 책 번호 찾기
- 아이디로 회원 정보 찾기

set

`set`은 중복을 허용하지 않는 자료구조이다.

```
set<int> numbers;  
numbers.insert(10);  
numbers.insert(10);  
numbers.insert(20);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - STL 개념정리	date

왜 필요한가

중복 없는 값만 저장하고 싶을 때 필요하다.

예시:

- 중복 없는 학생 번호
- 방문한 값 기록
- 고유한 단어 목록

sort

`sort()`는 데이터를 정렬하는 알고리즘이다.

```
vector<int> v = {5, 2, 4, 1, 3};  
sort(v.begin(), v.end());
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

결과 결과

```
1 2 3 4 5
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 필요한가

정렬은 데이터를 보기 좋게 만들 뿐 아니라, 이진 탐색이나 순위 계산처럼 다른 알고리즘을 사용하기 위한 준비 작업이 된다.

구조체 정렬

```
struct Book {  
    int id;  
    string title;  
};  
  
bool compare(Book a, Book b) {  
    return a.title < b.title;  
}  
  
sort(books.begin(), books.end(), compare);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 필요한가

단순 숫자 정렬이 아니라 책 제목순, 점수순, 날짜순처럼 원하는 기준으로 정렬해야 하는 경우가 많다. 비교 함수를 사용하면 정렬 기준을 직접 정할 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - STL 개념정리	date

iterator

iterator는 컨테이너의 원소를 가리키는 도구이다.

```
vector<int> v = {10, 20, 30};

for (vector<int>::iterator it = v.begin(); it != v.end(); it++) {
    cout << *it << endl;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 필요한가

STL 컨테이너는 종류가 다양하다. iterator를 사용하면 컨테이너 내부 구조가 달라도 비슷한 방식으로 원소를 순회할 수 있다.

C 도서관 프로그램과 STL

이번 도서관 프로그램은 C로 작성하기 때문에 STL을 사용하지 않는다. C에서는 배열, 문자열 함수, 직접 정렬, 직접 탐색을 사용해야 한다.

C++로 작성한다면 다음처럼 STL을 사용할 수 있다.

```
vector<Book> books;
sort(books.begin(), books.end(), compare);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

최종 정리

STL은 C++에서 자료구조와 알고리즘을 쉽게 사용하기 위해 필요하다. 직접 구현하면 오래 걸리고 실수하기 쉬운 기능을 표준 라이브러리로 제공한다. `vector`는 크기가 변하는 배열, `stack`은 후입선출, `queue`는 선입선출, `map`은 key-value 저장, `set`은 중복 없는 저장, `sort`는 정렬에 사용된다.

확인 질문

- STL은 왜 필요한가?
- vector와 배열의 차이는 무엇인가?
- stack과 queue는 어떤 상황에서 필요한가?
- map은 왜 사용하는가?
- set은 왜 사용하는가?
- sort가 필요한 이유는 무엇인가?

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - STL 개념정리	date

어디에 사용하는가

STL은 C++에서 데이터를 편하게 다룰 때 사용한다. 직접 배열, 연결 리스트, 정렬 함수를 만들지 않고 이미 만들어진 도구를 가져다 쓰는 것이다.

게임 예시:

<code>`vector`</code>	플레이어 인벤토리 아이템 목록
<code>`stack`</code>	되돌리기, 이전 화면으로 돌아가기
<code>`queue`</code>	접속 대기열, 몬스터 행동 순서
<code>`map`</code>	아이템 이름과 가격 저장
<code>`set`</code>	이미 방문한 맵 위치 저장
<code>`sort`</code>	점수 순위 정렬

도서관 예시:

<code>`vector<Book>`</code>	책 목록 저장
<code>`map<string, Book>`</code>	제목으로 책 찾기
<code>`set<int>`</code>	중복 없는 책 번호 관리
<code>`sort()`</code>	제목순 또는 번호순 정렬

어떻게 사용하는가

데이터를 여러 개 저장해야 하면 먼저 어떤 자료구조가 맞는지 고른다. 순서대로 저장하고 인덱스로 접근하고 싶으면 ``vector``, 먼저 들어온 순서대로 처리하고 싶으면 ``queue``, 이름으로 값을 찾고 싶으면 ``map``, 중복을 막고 싶으면 ``set``을 사용한다.

비유

STL은 공구함과 비슷하다. 못을 박을 때 망치를 직접 만들지 않고 공구함에서 꺼내 쓰듯이, 정렬이나 자료구조도 직접 만들지 않고 STL에서 꺼내 쓰는 것이다.